

Greenplum



Содержание

1 Greenplum

01

Greenplum



Greenplum

Greenplum – это распределенная аналитическая БД, в основе которой лежит PostgreSQL. Greenplum был создан для работы с большими объемами данных в режиме параллельной обработки запросов и использует MPP архитектуру.

Немного истории.

2005 год — первый выпуск технологии одноименной фирмой в Калифорнии(США);

2010 год — корпорация EMC поглотила компанию Greenplum, продолжив работу над проектом;

2011 год — корпорация EMC выпустила для всеобщего пользования бесплатную версию Greenplum Community Edition;

2012 год — корпорация Pivotal приобрела продукт EMC Greenplum Community Edition, продолжая далее развивать его под свои брендом;

2015 год — компания Pivotal опубликовала исходный код СУБД Greenplum под свободной лицензией Apache;

2020 год — корпорация VMware приобрела компанию Pivotal, которая была вендором GP. С этого момента open-source MPP-Субд коммерциализируется под торговой маркой VMware Tanzu Greenplum

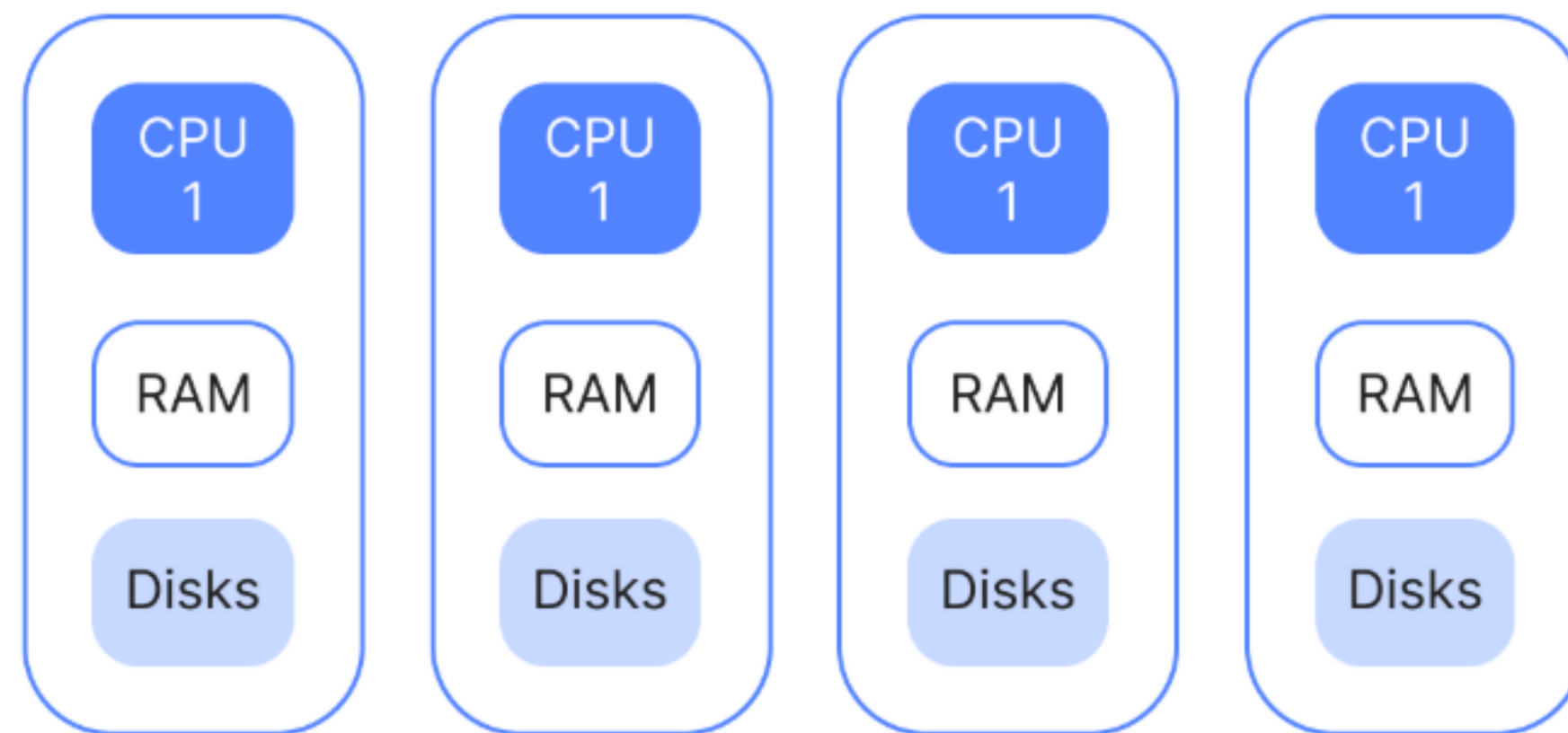
2024 год — переход проекта в архив, VMware Tanzu Greenplum развитие, как коммерческого продукта

Greenplum

SMP
(симметричная многопроцессорная архитектура)



MPP
(массивно-параллельная архитектура)



Greenplum и PGSQL

Сходства:

- Синтаксис SQL, объекты БД и встроенные функции;
- Структура системного каталога;
- Механизм реализаций транзакций;
- Ролевая модель;
- Работа драйвера для PGSQL.

Greenplum и PGSQL

Отличия:

- Специфичные изменения в системном каталоге и функциях (например, информация о том, как лежат данные на сегментах);
- Устаревшая версия PGSQL в актуальном GP;
- Механизмы переписаны под MPP-архитектуру.

Характеристики

Распределенная архитектура: GP использует разделение данных между несколькими узлами. Это позволяет эффективно обрабатывать большие объемы данных с высокой параллельностью. Каждый узел хранит свою часть данных, и запросы выполняются параллельно на всех узлах, что позволяет сократить время обработки.

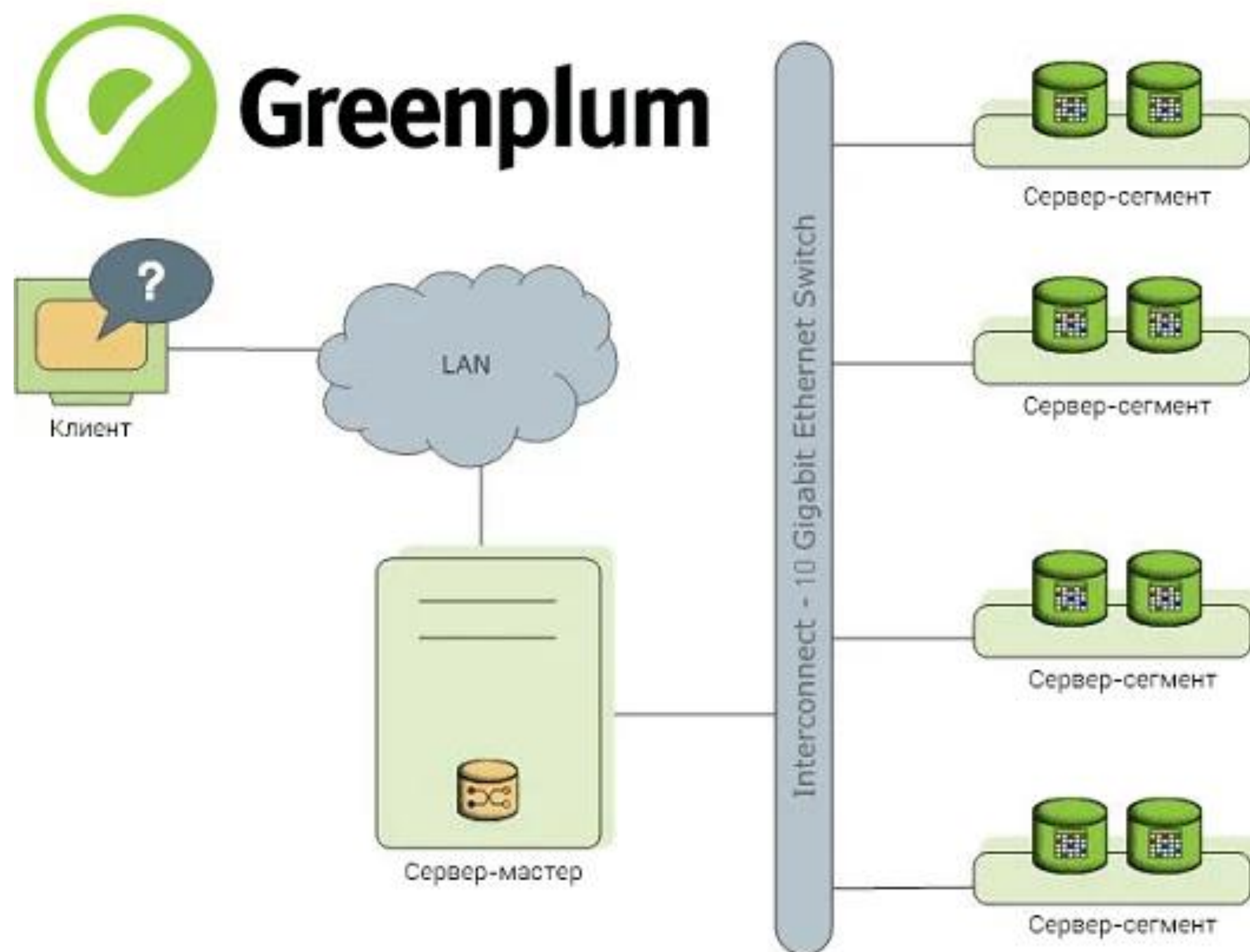
Репликация и отказоустойчивость: GP поддерживает репликацию данных между узлами для обеспечения отказоустойчивости. Если один узел выходит из строя, данные можно восстановить с другого узла.

Ядро PostgreSQL: Greenplum построен на PGSQL, что дает ему совместимость с существующими инструментами и приложениями, которые используют PGSQL. Он также поддерживает SQL и все типичные возможности PGSQL, индексы, расширения, партиции и т.д.

Поддержка колоночной системы хранения: Greenplum поддерживает колоночное хранение, что помогает ускорить аналитические запросы.

Инструменты для работы с большими данными: Greenplum поддерживает интеграцию с такими инструментами, как Apache Hadoop, Apache Spark и другие технологии Big Data.

Архитектура

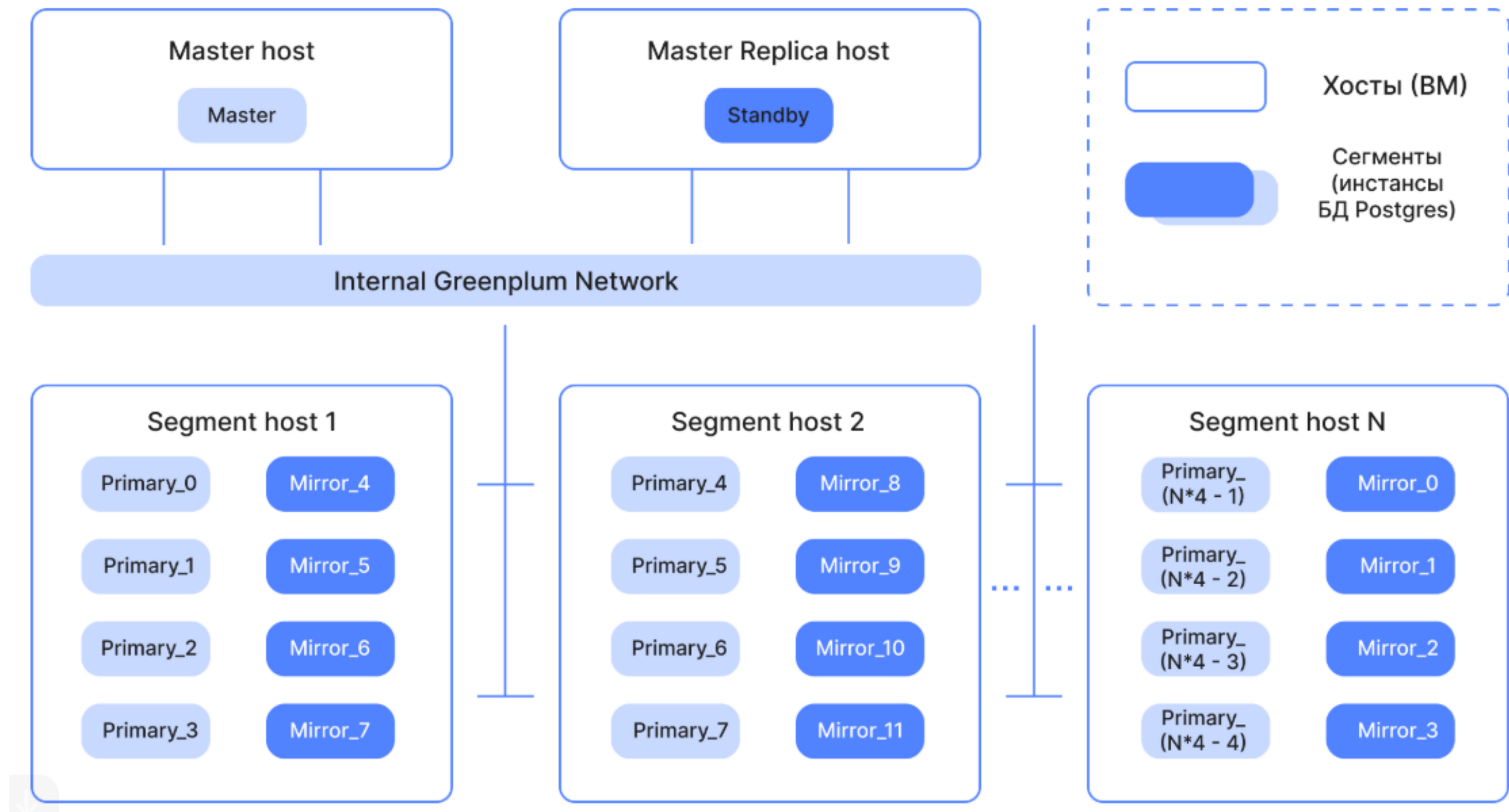


Архитектура и принципы работы

Архитектура

- **Мастер-сервер** (Master host, coordinator), где развернут главный инстанс PostgreSQL (**Master instance**). Он является точкой входа в Greenplum и представляет собой экземпляр базы данных, к которому подключаются клиенты, отправляя SQL-запросы. Мастер координирует свою работу с сегментами — другими экземплярами базы данных в этой Big Data системе.
 - Не хранит пользовательские данные(только метаданные).
 - Работает как точка входа для клиентов и приложений.
- **Резервный мастер** (Secondary master instance, standby) — инстанс PostgreSQL, который вручную включается в работу при отказе основного мастера.
- **Сервер-сегмент** (Segment host), который хранит и обрабатывает данные. Обычно 1 хост-сегмент содержит 2–8 сегментов Greenplum. Точное число экземпляров базы данных на хосте зависит от его характеристики (ЦП, память, жесткие диски), а также от сетевых интерфейсов и рабочих нагрузок. Сегменты Greenplum представляют собой инстансы PostgreSQL и делятся на основные (primary) и зеркальные (mirror). Каждая ячейка сегментов Greenplum — это независимая база данных PostgreSQL, где хранится часть данных. **Primary-сегмент** обрабатывает локальные данные, отдавая результаты мастеру. Каждому primary-сегменту соответствует свое зеркало (Mirror segment instance) — инстанс PostgreSQL, который автоматически включается в работу при отказе primary.

Архитектура



Мастер-сервер

Мастер-сервер строит план запросов:

- **Анализатор запросов (Parser)**

Мастер разбирает SQL-запросы, преобразует их в дерево операций (Abstract Syntax Tree).

- **Оптимизатор (Optimizer)**

Выбирает наиболее эффективный способ выполнения запроса;

Учитывает особенности распределения данных на сегментах;

Использует Cost-Based Optimization — на основе статистики таблиц.

- **Генератор плана (Plan Executor)**

Разбивает запрос на задачи, распределяет их по сегментам;

Передаёт slices(подзадачи) сегментам через interconnect.

Сегментный-сервер/сегмент

Сегментные серверы — это рабочие узлы, на которых хранятся и обрабатываются данные. Их и масштабируют для повышения производительности Greenplum:

- Каждый сегментный-сервер содержит один или несколько сегментов;
- Каждый сегмент функционирует, как независимая PGSQL БД.

Сегмент — это логическая и физическая часть данных, расположенная на сегментном сервере.

- Данные распределяются между сегментами с использованием хэшей или механизма broadcast;
- Распределение определяется с помощью distribution keys.(т.е. добавляя эту настройку в таблицу, вы говорите, по какому ключу данные будут распределяться(физически) по сегментам).

Принципы параллельной обработки в Greenplum

Выполнение запросов производится на первичных сегментах. Всегда задействуются все сегменты — пользователь не может задать выполнение запроса по части сегментов.

Сегменты GP работают синхронно, обеспечивая параллелизацию обработки каждого запроса. Каждый запрос распараллеливается на столько процессов, сколько есть первичных сегментов в кластере.

Distribution Key.

Distribution Key (ключ распределения) — это одна из важнейших концепций в Greenplum, которая определяет, как данные распределяются между сегментами в кластере. Правильный выбор ключа распределения может существенно повысить производительность, минимизируя сетевой трафик и обеспечивая балансировку нагрузки.

То есть ключ распределения, это столбец или набор столбцов в таблице, по которым Greenplum решает, как распределять строки этой таблицы по сегментам.

Типы дистрибуции:

- Hash distribution
- Round-Robin Distribution
- Replicated Distribution

Hash Distribution.

Наиболее частый способ распределения данных в Greenplum.

- **Что делает:**

Строки таблицы распределяются по сегментам на основе хеш-функции, которая применяется к значениям в выбранных колонках в Distribution Key.

- **Когда использовать:**

1. Если часто выполняются JOIN-операции с другими таблицами, и нужно минимизировать сетевой трафик между сегментами.
2. Когда операции в запросах зависят от значений в колонке (например, выборка данных по конкретному значению или диапазону).

- **Преимущества:**

1. Легко масштабируется по мере роста данных.
2. Минимизирует неравномерное распределение данных.

Round-Robin Distribution.

В этом случае строки таблицы равномерно распределяются по сегментам, независимо от их значений.

- **Что делает:**

Строки добавляются поочередно на сегменты, без привязки к значению в колонке.

- **Когда использовать:**

1. Когда таблица будет использоваться в основном для чтения или для общих агрегаций, где конкретная схема распределения не имеет значения.
2. Когда вы хотите избежать неравномерности в распределении на сегментах, но не ожидаете, что будут часто выполняться JOIN-операции по определенным колонкам.

Replicated Distribution.

В этом случае копия всей таблицы хранится на каждом сегменте. Это позволяет сократить время выполнения запросов, если таблица маленькая или если она часто используется для JOIN'ов.

- **Что делает**

Таблица или часть данных реплицируется на каждом сегменте. В случае JOIN-операций, если одна из таблиц маленькая, все сегменты имеют доступ к ее копиям, что ускоряет выполнение.

- **Когда использовать:**

1. Когда таблица небольшая(справочники, словари)
2. Если вы хотите ускорить выполнение JOIN'ов между большой таблицей и маленькой реплицированной.

DDL Distribution

Round-robin distribution

```
CREATE TABLE tt_table (  
    uuid text,  
    some_value text  
)  
  
WITH (  
    appendonly=true,  
    blocksize=32678,  
    orientation='column',  
    compresstype=zstd,  
    compresslevel='1'  
)  
  
DISTRIBUTED RANDOMLY;
```

Replicated distribution

```
CREATE TABLE tt_table (  
    uuid text,  
    some_value text  
)  
  
WITH (  
    appendonly=true,  
    blocksize=32678,  
    orientation='column',  
    compresstype=zstd,  
    compresslevel='1'  
)  
  
DISTRIBUTED REPLICATED;
```

Hash distribution

```
CREATE TABLE tt_table (  
    uuid text,  
    some_value text  
)  
  
WITH (  
    appendonly=true,  
    blocksize=32678,  
    orientation='column',  
    compresstype=zstd,  
    compresslevel='1'  
)  
  
DISTRIBUTED BY (uuid);
```

Heap. Виды таблицы.

Тип хранения данных по умолчанию в Greenplum. В этом типе данные хранятся в строках, и записи могут быть вставлены в любое место в таблице. В результате записи могут быть не упорядочены, что может приводить к фрагментации данных с течением времени. Однако это подходит для операций, которые требуют частого обновления и удаления строк.

Особенности:

- **Простота:** Стандартный и наиболее универсальный тип хранения
- **Операции вставки и удаления:** Хорошо поддерживает операции вставки, удаления и обновления данных.
- **Фрагментация:** Время от времени таблица может фрагментироваться, что может повлиять на производительность запросов.
- **Поддержка индексов:** Работает с большинством типов индексов, включая B-tree и GIN.

Когда использовать:

- **Динамичные данные:** Подходит для таблиц с данными, которые часто обновляются или удаляются
- **Малые и средние объемы данных:** Для таблиц, которые не требуют особо высокой производительности при чтении больших объемов данных.
- **Частые операции на запись:** Если данные часто изменяются и часто выполняются операции обновления и удаления.

Append-Optimized(AO). Виды таблицы.

Это тип хранения, специально оптимизированный для сценариев, когда данные только добавляются в таблицу, но не обновляются или не удаляются часто. Эти таблицы используют эффективный метод записи данных, чтобы ускорить операции вставки, уменьшить фрагментацию и повысить производительность при выполнении аналитических запросов. В отличие от типа Heap, таблицы с хранением АО не позволяют обновлять или удалять строки напрямую — любые изменения приводят к вставке новых строк.

Особенности:

- **Оптимизация для чтения:** Значительно ускоряет операции чтения за счет сжатия данных и меньшей фрагментации.
- **Высокая производительность при вставке:** Оптимизирован для сценариев, где данные добавляются, но не изменяются.
- **Недоступность операций обновления/удаления:** Таблицы АО не поддерживают прямое обновление или удаление строк, данные только добавляются. Для удаления используется метод логической очистки (VACUUM).
- **Сжатие данных:** таблицы АО используют эффективное сжатие данных, что позволяет значительно уменьшить объем данных на диске.

Append-Optimized(AO). Виды таблицы.

Когда использовать:

- **Чтение больших объемов данных** — хорошо подходит для аналитических запросов, где данные часто добавляются, но редко обновляются или удаляются;
- **Логирование и агрегированные данные** — идеально для таблиц, которые записывают большие объемы данных (например, логи, результаты обработки и т.д.), где записи в таблице происходят в режиме append
- **Применение для исторических данных** — хорошо подходит для хранения исторических данных или данных, которые добавляются на постоянной основе.

Сжатие в Greenplum

Эффективное сжатие данных позволяет снижать потребление памяти и повышать производительность SQL-запросов. Greenplum предлагает несколько методов сжатия данных, снижения затрат на хранения и повышения производительности запросов.

В Greenplum доступны два типа сжатия в БД:

- сжатие на уровне таблицы;
- сжатие на уровне столбца.

Ориентация таблицы	Доступные типы сжатия	Поддерживаемые алгоритмы
Строковая	На уровне таблицы	ZLIB и ZSTD
Колоночная	На уровне столбца и таблицы	RLE_TYPE, ZLIB и ZSTD

Что есть в планах запросов.

Gather motion — означает объединение результатов выполнения запросов со всех сегментов в один поток (как правило, на мастер).

Broadcast motion — каждый сегмент отправляет свою копию данных на другие сегменты. В идеальной ситуации бродкаст происходит только для словарей, списков (маленькие таблицы).

Redistribution motion — для соединения больших таблиц, распределенных по разным ключам, выполняется перераспределение с целью выполнения соединений локально. Для больших таблиц может быть достаточно затратной операцией.

Минусы

Сложность настройки и управления: В отличие от однородных СУБД, Greenplum требует более сложной настройки и управления, особенно при увеличении числа узлов и объема данных.

Необходимость в высокопроизводительном оборудовании: Для достижения максимальной производительности Greenplum требует мощных серверов и сетевой инфраструктуры.

Ограниченная поддержка транзакций: Хотя Greenplum поддерживает транзакции, его основные сильные стороны связаны с аналитикой, а не с OLTP. Поэтому он может не подходить для сценариев, где нужна высокая частота транзакционных операций.